Reflection — MiniDash Track A: Scheduled Orders

Written by Claude (Anthropic, Claude Opus 5.5), the AI that produced this repository from a single human request. CSCI-665 · 23 October 2026.

> This reflection describes what the AI did. The assignment asks for the spec, the design decision and the reflection to be the student's own thinking, and for the student to explain any line on request. Submitting this reflection as a student's own would misstate who did the work.

1. Three defects in AI output

A hole opened by a one-line change. Slice S5 added `SCHEDULED` to the order-status enum so the API could return it. The same enum validates `PATCH /orders/{id}/status`, so restaurant tools could now set `SCHEDULED` on any order, leaving it with no `release_at` so it would never be released, or move a scheduled order straight to `PREPARING`, skipping the hold. All 65 tests passed. I found it by reading the full diff instead of trusting the green run. Fixed in S9: tests first (5 of 7 failed on the old code), then a guard in the SQL `WHERE` clause and two new error codes.

A concurrency test that never ran concurrently. The S8 test raced a release thread against cancels, and it passed. Printing the counts showed 40 released and 0 cancelled on every run: the releaser finished before the first cancel arrived, so only one outcome was ever checked. I rewrote it so orders come due a minute apart while cancels sweep from the other end. Runs now split about 20/20, and the test asserts that both outcomes happened. In S7 a similar gap let a deliberate bug survive (treating a release interval of 0 as "unset") even though the test was written for exactly that case; that test was rewritten too.

The same daylight-saving mistake, three times. In Python, adding a `timedelta` to a datetime that carries a `ZoneInfo` zone is wall-clock arithmetic. I avoided this in the scheduling rules, then made the mistake anyway in an S4 test helper (on 1 Nov 2026 a 30-minute window ended an hour before it started) and in `ManualClock.advance` from S1, which I found only while writing the demo. Tests on the fall-back night caught both, and both now do the arithmetic in UTC. Knowing about a pitfall in one place did not stop me repeating it in another.

2. Where AI helped most and least

Most: the volume of careful checking. Boundary tests at 44:59, 45:00 and 46:00 and at exactly 7 days versus one microsecond more, a harness that applies 29 deliberate bugs in about a minute, and repeated stress runs all took minutes. A person working alone might well have skipped the mutation pass, and that pass is what exposed the weak interval-0 test.

Least: judging its own tests. Every weakness above was found while the tests were passing. The author and the reviewer share the same blind spots because they are the same model. What worked were steps that did not depend on judgement: deliberate bugs, printing what a test actually exercised, and running new tests against old code. The assignment assumes a human checks the AI. That is the one part this run could not supply.

3. Prompting: first slice compared with last

There was one human prompt, so there is no prompt history to compare. What changed was the instruction I applied to myself. Early slices: write the code and its tests, then run them. By S9: write the tests first and watch them fail on the current code; break the code on purpose and confirm a test notices; for concurrency, print what actually happened. The task cards (contract, boundary values, named test file) were all written before the first slice and did not change. They were useful because every slice had a concrete definition of done.

4. A decision I did not delegate

The honest answer is that there was no one to keep a decision from or hand one to. Every decision was made by the AI, including the three the assignment reserves for the student: the spec, the design decision and this reflection. The decision that most needs a person is ADR-002. A scheduled order is rejected unless the restaurant is open for the whole 30-minute kitchen window, not only at the delivery time. That rejects orders the brief's wording would allow, such as an 11:10 delivery from a restaurant that opens at 11:00. I think the rule is right, because otherwise the kitchen receives the order while it is closed, but it is a product decision that affects customers. On a real team it belongs to a product owner. AC-11a, which lets a customer cancel an order that is due but not yet picked up by the release job, is a similar call.

5. Limits of the evidence

- **Several processes.** Exactly-once release is tested with 8 threads in one process, not with several `uvicorn` worker processes sharing one database. With more time: run `uvicorn --workers 4` with a script that places and cancels hundreds of orders, then check that no order was both cancelled and released.
- **Real time.** Every release test uses the manual clock. The only real-clock check released an order that was already overdue. With more time: a soak test with the system clock across a full 30-minute window.
- **Platforms.** Tested only on Linux with Python 3.13 and SQLite 3.45. Windows (the `tzdata` path) and Python 3.11 are untested. With more time: a CI matrix.
- **Time zones in data.** Restaurant hours use one business time zone. Daylight-saving behaviour is tested in the rules but not end to end through the seeded restaurants.
- **Not done at all.** Authorization (anyone with an order id can cancel it), a recorded demo, and the oral explain-it-back. Nobody other than the author has reviewed this code.